

Harris Corner Detector

Mohammadreza Abolhassani

1 Project Overview

This project implements a Harris corner detection application in Python. The goal of the project is to detect visually important corner points in grayscale images using the Harris Corner Detection algorithm.

The project was developed by Mohammadreza Abolhassani and includes a graphical user interface built with Tkinter. The GUI allows the user to load an image, adjust corner detection parameters, preview the selected image, and visualize the final detection results.

Corners are important in computer vision because they are stable and distinctive image features. They can be used in applications such as object recognition, image matching, motion tracking, feature extraction, and 3D reconstruction.

2 Application Interface

The application provides a simple interface where the user can adjust the Harris detector parameters before running the algorithm.

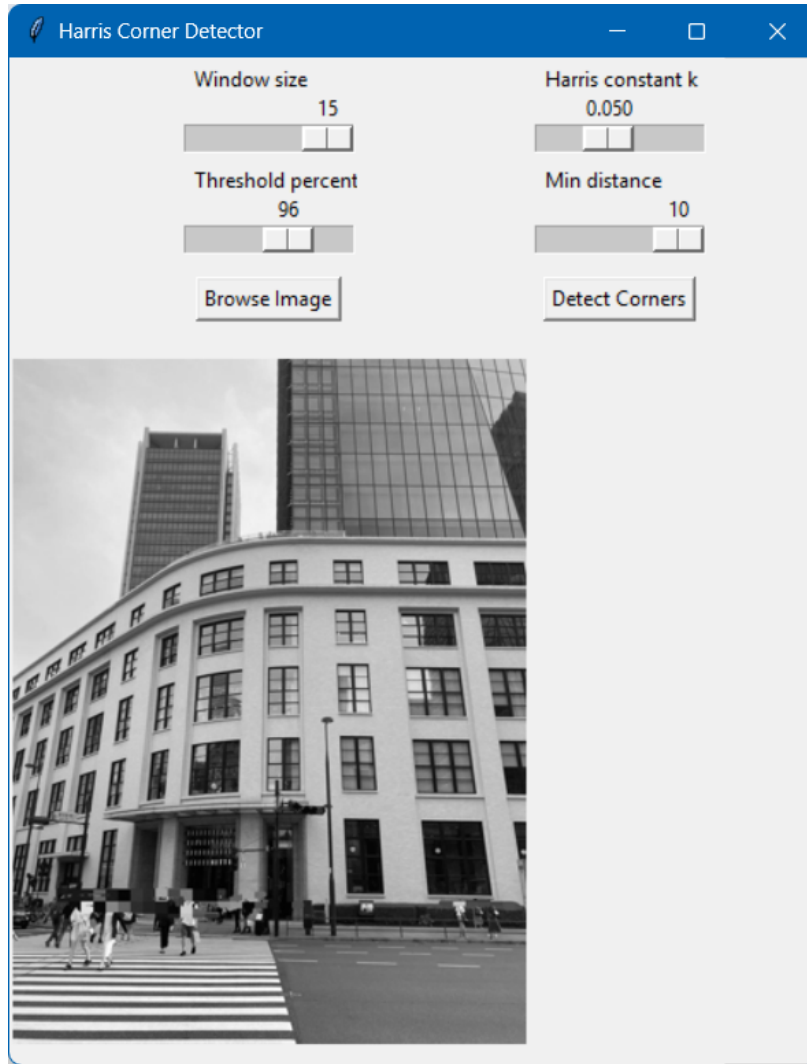


Figure 1: Tkinter GUI for the Harris Corner Detector

The GUI includes sliders for the following parameters:

- Window size
- Harris constant k
- Threshold percentile
- Minimum distance between detected corners

3 Project Pipeline

The corner detection pipeline consists of the following steps:

1. Load the input image as a grayscale image.

2. Compute horizontal and vertical image gradients.
3. Compute products of image gradients.
4. Build the second moment matrix for each pixel.
5. Compute the Harris response value.
6. Threshold weak corner responses.
7. Apply non-maximum suppression.
8. Display the detected corners on the original image.

4 Image Gradients

The first step in the algorithm is computing the image gradients in the horizontal and vertical directions. These are represented as:

$$I_x$$

and

$$I_y$$

where I_x measures changes in intensity in the horizontal direction and I_y measures changes in intensity in the vertical direction.

In this project, Sobel filters are used to compute these gradients.

5 Gradient Products

After computing the gradients, the algorithm calculates the following products:

$$I_x^2$$

$$I_y^2$$

$$I_x I_y$$

These values describe how image intensity changes around each pixel.

6 Second Moment Matrix

For every pixel, the gradient products are summed over a local window. This creates the second moment matrix:

$$M = \begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix}$$

where:

$$S_{xx} = \sum I_x^2$$

$$S_{yy} = \sum I_y^2$$

$$S_{xy} = \sum I_x I_y$$

The matrix M describes the local structure around a pixel.

7 Harris Response

The Harris response is computed using the equation:

$$R = \det(M) - k(\text{trace}(M))^2$$

where:

$$\det(M) = S_{xx}S_{yy} - S_{xy}^2$$

and

$$\text{trace}(M) = S_{xx} + S_{yy}$$

The value of R tells us what type of region the pixel belongs to:

- If R is large and positive, the region is likely a corner.
- If R is large and negative, the region is likely an edge.
- If R is close to zero, the region is likely flat.

8 Thresholding

After computing the Harris response map, weak responses are removed using a percentile threshold. This keeps only the strongest corner candidates.

For example, if the threshold percentile is set to 97, only the strongest 3 percent of responses are kept.

9 Non-Maximum Suppression

Thresholding alone may still leave many nearby points around the same corner. To solve this, non-maximum suppression is applied.

This step keeps only the strongest local maximum within a specified minimum distance. As a result, each physical corner is represented by fewer and cleaner detected points.

10 Results

The output window displays the intermediate and final results of the algorithm.

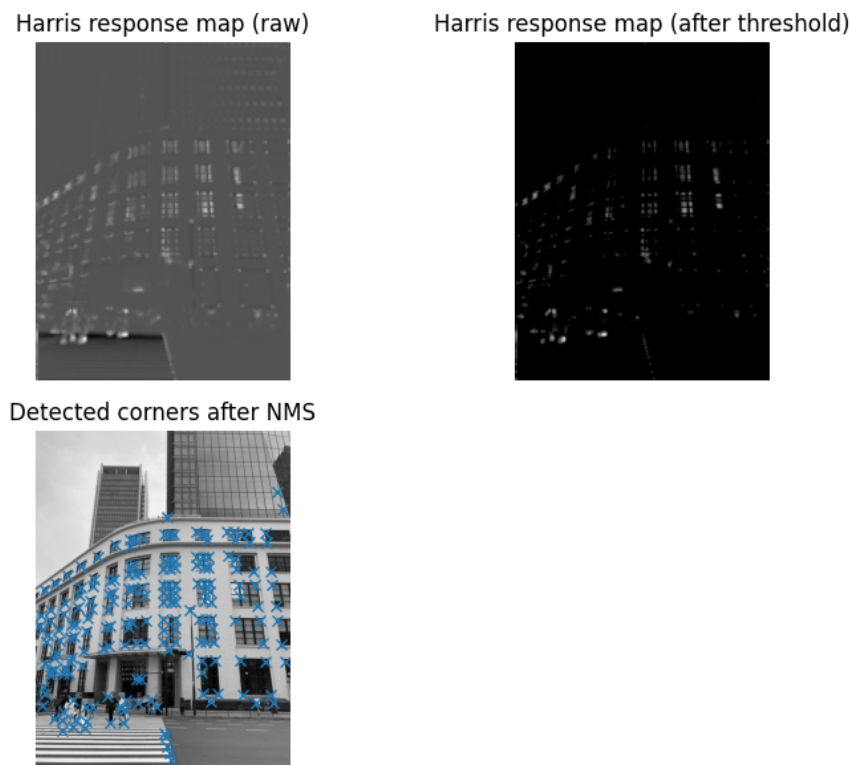


Figure 2: Harris corner detection results

The result includes:

- The raw Harris response map
- The thresholded Harris response map
- The original image with detected corners marked

11 Conclusion

This project demonstrates the complete Harris corner detection pipeline from scratch using Python. It shows how image gradients can be used to identify regions with strong intensity changes in multiple directions.

The project also provides an interactive GUI, making it easier to experiment with different parameters and observe how they affect the detected corners.